

Base de datos de Territorio Preparado: informe de montaje en Supabase

Proyecto: Territorio Preparado, RETO-01 Cali Activa (Hackathon Smart City Expo Cali 2026).

Fecha: 4 de octubre de 2026. **Quién lo hizo:** William Ortiz, con Claude Code.

Estado: la base está **montada en Supabase dev**: 12 migraciones aplicadas y la API expone solo `api`, verificado contra el proyecto real. Los datos reales están cargados y verificados. Faltan el MFA, el hook del token y cambiar la contraseña de la base (sección 8).

Este documento no contiene contraseñas, claves ni datos personales.

1. Resumen

Qué	Resultado
Modelo de datos (Hito 1)	6 esquemas, 31 tablas, 24 vistas, 32 políticas de seguridad por fila (RLS) y 35 permisos por rol
SQL (Hito 1)	12 migraciones en <code>db/supabase/migrations/</code> , con el formato de la CLI de Supabase
Datos reales (Hito 2)	Script de carga con verificación sha256: 22 comunas, 342 barrios, 2.991 espacios, 660 zonas de amenaza y 182 juntas de acción comunal
Pruebas	69 pruebas del SQL y 17 de la carga, todas en verde (<code>cd db && npm test</code>)
Repositorio	Pasado a privado ; 3 integrantes invitados con permiso de escritura; pull request #2 abierto hacia main
Supabase	Proyecto dev (ref <code>rqxltixtsiqsaxpdae</code> , región EE. UU. este , <code>us-east-1</code>), 12 de 12 migraciones aplicadas y datos reales cargados

2. Qué se hizo, paso a paso

- SQL del Hito 1.** Se tradujo el modelo de `db/docs/` a 11 migraciones: esquemas, tablas con sus reglas de integridad, auditoría encadenada, catálogos, permisos, RLS, vistas y funciones de la API, y privilegios. Se probaron en PGlite, que es PostgreSQL 18 con PostGIS 3.6 corriendo en Node, porque en el equipo no hay Docker.
- Carga de datos reales (Hito 2).** Se escribió `db/scripts/cargar_datos_reales.mjs` y la migración 12 (`id_fuente` para recargar las juntas sin duplicarlas). Se comprobó que los cruces de amenaza calculados por PostGIS coinciden **uno a uno** con los que precalculó la app: 884 espacios con cruce de inundación y 1.369 con cruce sísmico.
- Repositorio privado.** El repo `leonidas452528/hackaton-camaracomercio2026` pasó a privado. La demo de GitHub Pages sigue publicada, porque la cuenta es GitHub Pro.
- Integrantes.** Se invitó con permiso **Write** a `helynecheverry` (Herlin), `PabloEARangoM` (Pablo) y `bryanmavi` (Bryan). Bryan ya aceptó; Herlin y Pablo, todavía no.
- Pull request.** La rama `db/esquema-inicial` se subió y quedó como PR #2 hacia main, para que la

revise el responsable de la base de datos.

6. **Proyecto en Supabase.** Se creó el proyecto dev con la cuenta de GitHub. Ref: `rqxltixtsiqsaxapdue`; URL de la API: `https://rqxltixtsiqsaxapdue.supabase.co`.
7. **CLI de Supabase.** Se instaló como dependencia de desarrollo de db/ (versión 2.119.0), así todo el equipo usa la misma versión. Se generó `db/supabase/config.toml`, con `api` como único esquema expuesto, el hook del token, MFA por aplicación (TOTP) y sin `seed.sql`.
8. **Enlace.** `npx supabase login` y `npx supabase link --project-ref rqxltixtsiqsaxapdue` se corrieron en una terminal normal. Dentro de Claude Code el login falla porque no hay una terminal interactiva.
9. **Migraciones.** `npx supabase db push --dry-run` mostró las 12 migraciones pendientes y nada más. `npx supabase db push` las aplicó sin errores, y `npx supabase migration list` confirma que lo local y lo remoto coinciden.
10. **Exposición de la API.** La primera consulta a la API respondió `PGRST106` (esquema no expuesto). William dejó `api` como esquema expuesto en *Project Settings* → *Data API*, y se repitió la verificación como público (`anon`), con la clave pública:

Consulta como público (anon)	Resultado en Supabase
<code>api.amenazas</code> , <code>servicios</code> , <code>funciones_espacio</code> , <code>entidades</code> , <code>responsabilidades</code> , <code>parametros_reglas</code>	5, 12, 7, 24, 4 y 5 filas: los catálogos sembrados
<code>api.espacios</code>	0 filas: falta la carga de datos reales
<code>api.decisiones</code> y <code>api.auditoria</code>	Bloqueado (42501): solo con sesión y rol
Función de escritura <code>registrar_medicion</code>	Bloqueada (42501): el público no escribe
Esquemas <code>ops</code> y <code>public</code>	No expuestos (PGRST106)

11. **Carga de los datos reales.** La primera conexión falló por verificación TLS (`SELF_SIGNED_CERT_IN_CHAIN`): Supabase firma sus certificados con su propia CA. No se desactivó la verificación. Se descargó la CA raíz pública de Supabase ("Supabase Root 2021 CA", vigente hasta 2031, huella SHA-256 `80:70:25:AD...:CA:FA`) a `db/certs/` y se comprobó con `openssl` que el pooler presenta un certificado válido para su nombre (`Verify return code: 0`). La carga se corrió con `sslmode=verify-full`:

Verificación en Supabase	Resultado
Archivos crudos (sha256 y conteo)	9 de 9 coinciden con el manifiesto
Cargado	22 comunas, 342 barrios, 2.991 espacios, 660 zonas de amenaza y 182 JAC
API pública (anon, conteo exacto)	espacios 2.991, comunas 22, barrios 342, zonas_amenaza 660
Cruces de PostGIS frente a los de la app	Inundación 884 y sísmico 1.369: 0 diferencias
Ficha pública de epou-8413	Nombre, comuna y área de la fuente; baños y capacidad en null; disponibilidad "Por confirmar"

3. Qué quedó en la base

Esquema	Contenido	¿Lo ve la API?
ref	9 tablas de catálogos: 5 amenazas, 12 servicios y tareas, 7 funciones del espacio, 24 entidades, 5 parámetros de planeación, 4 filas de la matriz de responsabilidades y un protocolo de 11 pasos	No
geo	6 tablas de territorio (comunidades, barrios, espacios, zonas de amenaza, mediciones, organizaciones comunitarias) y 2 vistas (exposición y mediciones vigentes)	No
ops	9 tablas de operación (recomendaciones, decisiones, brechas, seguimientos, tareas, reportes comunitarios, retornos, lecturas IoT simuladas, notificaciones simuladas)	No
idn	5 tablas de identidad (perfiles, cargos, asignaciones, permisos por rol, autorizaciones de tratamiento)	No
aud	Eventos de auditoría y verificaciones de la cadena	No
api	22 vistas y 12 funciones: el único contrato con el frontend (db/docs/CONTRATO_API.md)	Sí

Todo lo institucional (entidades, matriz de responsabilidades, protocolo, funciones del espacio por amenaza) está marcado como **propuesta**: no lo ha validado la Secretaría de Gestión del Riesgo. No hay plazos ni cifras sin fuente. Los parámetros de baños, agua y superficie salen del Manual Esfera 2018.

4. Reglas que la base hace cumplir

- **La recomendación no reemplaza a la autoridad.** Solo `gestion_riesgo`, con verificación en dos pasos, registra una activación, y debe citar el acto administrativo. Si no sigue la recomendación, debe justificar.
- **No hay datos de personas.** Ninguna tabla guarda damnificados. Los reportes son conteos agregados, y los textos libres rechazan correos, teléfonos y números largos.
- **Desconocido no es cero.** Sin una medición verificada, el faltante queda vacío. Verificar exige evidencia y una persona distinta de quien reportó.
- **Cada quien ve lo suyo.** Las entidades ven sus brechas y tareas; las juntas, su zona; la consulta, solo agregados, con los conteos menores a 5 ocultos.
- **Cambios de cuenta inmediatos.** El rol se lee de la base en cada consulta, así que una suspensión o un traspaso de cargo surte efecto al instante.
- **Auditoría verificable.** Los eventos se encadenan con SHA-256 y no se pueden borrar ni modificar. Las pruebas incluyen una manipulación deliberada, que la verificación detecta.
- **Una sola puerta.** Ningún rol escribe directamente en las tablas: todo pasa por las funciones de `api`, que verifican permiso, alcance y MFA.

5. Verificaciones realizadas

Prueba	Resultado
69 pruebas de seguridad y flujo (PGlite)	Todas pasan
17 pruebas de carga con los datos reales	Todas pasan; la carga se puede repetir y se deshace completa ante

(PGLite)	una falla
Cruces de PostGIS frente a los de la app (PGLite y Supabase)	0 diferencias en 2.991 espacios
Huellas sha256 de los 9 archivos crudos de la IDESC	Coinciden con manifest.json
supabase db push --dry-run	Solo las 12 migraciones esperadas
supabase db push	12 de 12 aplicadas sin errores
supabase migration list	Lo local y lo remoto coinciden
API REST de Supabase como anon	Catálogos legibles; decisiones, auditoría y escrituras bloqueadas; ops y public no expuestos
Tiempos medidos en PGLite: una ficha, 2 ms; la lista de 2.991 espacios, 36 ms; todas las fichas completas, alrededor de 1 s.	

6. Archivos en el repositorio

Ruta	Qué es
db/supabase/migrations/	Las 12 migraciones (nunca se edita una ya aplicada: los cambios van en una migración nueva)
db/supabase/config.toml	Configuración de la CLI (sin secretos)
db/scripts/ cargar_datos_reales.mjs	Carga del Hito 2
db/certs/supabase-prod-ca-2021.crt	CA raíz pública de Supabase, para verificar TLS (no es un secreto)
db/tests/	Imitación mínima de Supabase para las pruebas, ejecutor y pruebas
db/docs/	Modelo, diccionario, roles, decisiones, contrato de la API, este informe y la guía de acceso del equipo
docs/cumplimiento/	Normas verificadas, entes decisores, red comunitaria y matriz normativa

7. Lo que NO está en el repositorio (y no debe estar)

La contraseña de la base de datos, el token de la CLI, la clave `service_role` o `secreta`, y las URL de conexión. El token de la CLI quedó solo en el equipo de William; la contraseña, en su gestor de contraseñas.

8. Pendientes para terminar el montaje

1. ☒ **Exponer solo api:** hecho y verificado (public responde "no expuesto").
2. ☐ **Activar el MFA TOTP:** *Authentication → Multi-Factor*.
3. ☐ **Activar el hook del token:** *Authentication → Hooks → Customize Access Token (JWT) Claims*, tipo Postgres, esquema idn, función custom_access_token_hook.
4. ☒ **Datos reales cargados y verificados.** Para recargar (es idempotente), con la contraseña escrita de forma oculta:

```
cd ~/hackathon-cali-2026/db
read -rs PGPASSWORD && export PGPASSWORD
DATABASE_URL="postgresql://postgres.rqxltixtsiqsaxapdue@aws-0-us-east-1.pooler.supabase.com:5432/postgres?sslmode=verify-full&sslrootcert=$PWD/certs/supabase-prod-ca-2021.crt" npm run cargar
unset PGPASSWORD
```

4b. ☐ **Cambiar la contraseña de la base**, porque quedó expuesta en el chat y en el historial de la terminal: *Project Settings → Database → Reset database password*. Después, volver a hacer `npx supabase link` en una terminal normal y borrar la línea del historial.

5. ☐ **Invitar al equipo en Supabase:** *Organization settings → Team → Invite member*, con el rol **Developer** (guía aparte: ACCESO_EQUIPO).
6. ☐ **Revisar y fusionar el PR #2.** Lo hace el responsable de la base de datos, que además responde las 10 decisiones abiertas de MODELO_DATOS.md §9.
7. ☒ **Hito 5, cuentas de demostración:** 13 cuentas creadas y verificadas (sección 9).
8. ☐ **Frontend:** pantallas de inicio de sesión e inscripción del factor TOTP.

9. Hito 5: cuentas de demostración

- **Migración 13:** `api.provisionar_perfil` y `api.asegurar_organizacion`. Solo las puede ejecutar `service_role`, y son idempotentes. Se aplicó con `supabase db push`.
- **Script `db/scripts/crear_usuarios_demo.mjs`** (`npm run cuentas-demo`): crea las cuentas con la Admin API de Supabase (correos `@example.org`, contraseñas aleatorias de 24 caracteres) y sus perfiles con cargo y organización. Después inicia sesión con cada una para verificarla.
- **Las contraseñas no están en el repo ni se imprimen:** quedan en `~/config/territorio-preparado/cuentas_demo_rqxltixtsiqsaxapdue.csv`, con permisos 600, en el equipo de William. Las claves de servicio se pasaron de la CLI al script sin mostrarse.
- **Organizaciones ficticias** para las cuentas comunitarias, en las comunas 06 y 05 y el barrio 0610: no se usan juntas reales.

Verificación en Supabase real	Resultado
Las 13 cuentas inician sesión y <code>api.mi_perfil</code> devuelve su rol	13 de 13
Claim <code>user_role</code> en el token (hook activo)	13 de 13
<code>gestion_riesgo</code> sin MFA intenta decidir	Bloqueado: "exige verificación en dos pasos (aal2)"
UAESP intenta decidir	Bloqueado

JAC A (comuna 06) declara en un espacio de la comuna 05 Bloqueado: "fuera de tu zona"

Reporte cuyos grupos de edad no suman el total Rechazado

consulta lee brechas y auditoría 0 filas; sí ve el resumen territorial (22 comunas)

superusuario ve las cuentas; gestiona usuarios sin MFA Ve 13; la gestión queda bloqueada

Auditor verifica la cadena de auditoría Correcta (68 eventos)

10. La app conectada a la base

- **Lectura:** la app (maqueta3d) lee el territorio desde el esquema `api` cuando tiene `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY`. Si no, o si la base no responde, usa los JSON estáticos y lo indica en el encabezado. La demo de GitHub Pages sigue en modo estático.
- **Paridad comprobada:** en PGLite, los 2.991 espacios, 22 comunas, 342 barrios y 660 zonas son idénticos a los JSON campo por campo. En Supabase real también, salvo las coordenadas del punto representativo: difieren como máximo en $1,4 \cdot 10^{-14}$ grados, porque la API redondea a 15 cifras.
- **Error encontrado y corregido:** la primera lectura desde Supabase paginaba con un orden repetido (`zona_id`), y eso duplica o salta filas entre páginas. Ahora todas las paginaciones usan un orden único.
- **Migración 14:** vista `api.mediciones_verificadas` y columna `orden_fuente` (la posición en el archivo de la IDESC), para que listas y mapa salgan en el mismo orden. Los datos se recargaron en Supabase.
- **Pestaña 06 Operación:** inicio de sesión, verificación en dos pasos (inscripción con código QR) y, según los permisos de `api.mi_perfil`, registrar decisiones con su acto administrativo (las necesidades salen de `planning.ts`), avanzar brechas, completar tareas, cerrar retornos y enviar reportes comunitarios.
- **Pruebas:** 23 unitarias y 22 de navegador de la app siguen pasando en modo estático. La prueba de humo contra Supabase real confirmó: encabezado "Base de datos"; UAESP inicia sesión y ve su perfil; la Secretaría ve la verificación en dos pasos, con la decisión bloqueada hasta completarla.
- **Mediciones (migración 15):** en Operación se declara una medición del espacio seleccionado; el formulario cambia según el tipo de dato, y la evaluación estructural es solo sí/no con fecha, sin concepto. La lista de pendientes trae los botones Verificar y Rechazar para quien puede validar ese atributo. Probado en Supabase real: la JAC A declaró 3 baños en `epou-9535` y no pudo validarlos; UAESP los verificó con evidencia.
- **Lo simulado no se publica como real:** la ficha pública (`api.espacio_ficha`) y el mapa excluyen las mediciones de las cuentas demo. En `epou-9535` la ficha sigue en `banos: null`, mientras `api.mediciones_verificadas` muestra el dato marcado `es_simulado: true`. En dev queda esa única medición de demostración.
- **TOTP en el plan gratuito:** comprobado en el proyecto con una inscripción de prueba que se borró al instante. El MFA por SMS sí exige plan Pro, pero no se usa.
- **Tiempo de carga:** bajó de unos 8,5 s a **1,3-1,8 s**, medido en el navegador contra Supabase real. Se logró con los cruces precalculados en una vista materializada que la carga refresca en la misma transacción (migración 16), y con las páginas pedidas en paralelo. La paridad con los JSON se mantiene. Falta probar en el navegador el registro de una decisión con MFA real.

11. Decisiones tomadas en esta sesión

- **Las pruebas corren en PGLite**, sin Docker ni Supabase local: cualquier integrante las corre con `npm test`.
- **Los catálogos van en migraciones**, no en `seed.sql`, porque el proyecto se arma solo con migraciones (ADR-0009).
- **La autorización lee la base, no el token**, para que la suspensión sea inmediata.
- **Se cargan las capas derivadas de la app**, no las crudas. Así los cruces son idénticos: la capa sísmica de la app usa 5 de los 16 polígonos de microzonificación, los que tienen susceptibilidad a licuación o corrimiento.
- **Los datos se cargan tal como vienen**. No se inventa ni se corrige en silencio: 52 espacios sin comuna, 23 juntas con un código de barrio que no está en la capa, y 3 huellas y 18 zonas con geometría no válida se cargan así y quedan reportadas.
- **La conexión verifica el certificado TLS** contra la CA pública de Supabase (`db/certs/`); no se desactiva la verificación.
- **Región:** el proyecto quedó en EE. UU. este (`us-east-1`). Esto resuelve en la práctica la decisión §9.10 para dev: EE. UU. figura en la lista de la SIC (Circular 005 de 2017) y Brasil no.